Ops installation guide

We can install Ops either by downloading the pre-built binary or compiling it from the source. In this article, we will use the pre-built binary of Ops for the sake of simplicity. For this demonstration we will be using Ubuntu 20.04 LTS as the host operating system.

Installation prerequisites - QEMU

To run Nanos, the QEMU hypervisor needs to be installed by executing the command given below:

```
$ sudo apt-get install qemu-system-x86
qemu-utils bridge-utils libvirt-daemon-
system libvirt-clients
```

We then have to install Ops as a non-root user. It can be installed as root as well, but that's not advised because of security implications.

```
$ curl https://ops.city/get.sh -sSfL | sh
```

Once Ops is installed, follow the on-screen instructions and run the command given below to check if Ops is working as expected:

```
$ ops version

Ops version: 0.1.31
Nanos version: 0.0
```

Note: Ops with QEMU may not work as expected in a nested virtualised environment, i.e., running Nanos unikernels with QEMU on top of a virtual machine. It may require additional configuration at the host level. In such a scenario, refer to the section "Deploying unikernels to the cloud'.

Ops configuration files

Providing a configuration file is not mandatory — arguments and attributes can be passed to the command directly. But this is not the most efficient way of running Nanos with Ops, as we may want to document the configuration for version control. The JSON configuration file is passed as a parameter and contains various attributes of code execution such as files to include, additional arguments, and environment variables. There are lots of configuration options or attributes; do refer to *https://docs.ops.city/ops/configuration* for more details.

A sample configuration file for a Node.js app is given below. Let us not deep dive into all the details here — we will focus on what's required to run our app.

```
{
     "Files": ["app.js"],
     "Dirs": ["src"],
     "Args": ["app.js "],
     "Env": {
       "NODE_DEBUG": "*",
       "NODE_DEBUG_NATIVE": "*"
     },
     "MapsDirs": {
       "src": "./"
     },
     "NameServers": ["192.168.1.1"],
     "RunConfig": {
       "Verbose": true,
       "Ports": [8080],
       "Memory": "1G"
     }
}
```

Ensure the configuration file is placed outside the directory named *src*. This can be named anything, but preference should be given to a meaningful naming convention as we will put our application code inside this directory. It's better to restructure our application directory to mimic the below-mentioned structure while uploading the code to a version control system like Git.

```
{application-staging}
└── src
      |- application files and libraries
└── config.json
```

Running an application inside the Nanos unikernel

It's time to put Nanos unikernel into action. We will run a Node.js application inside the unikernel using Ops. This walkthrough assumes that we have already installed Ops by following the steps mentioned earlier. The application that we will be deploying locally is a simple pocket calculator, the source code of which is available on GitHub. We will need Node.js on our host machine so that we can install the required application libraries and dependencies.

To install Node.js, execute the commands given below, which will install v16.x:

```
$ curl -sL https://deb.nodesource.com/
setup_16.x -o /tmp/nodesource_setup.sh
$ sudo bash /tmp/nodesource_setup.sh &&
sudo apt install nodejs
$ node -v
```

Next, we will have to check our application code and install its libraries:

```
$ git clone https://github.com/
actionsdemos/calculator.git calc-app
$ cd calc-app
$ npm install
```

The directory *calc-app*, where we checked our code, is the application staging area. But this will depend on the directory structure that is maintained on Git. In our case, we have our application code within the *src* directory; else, we would have created the *src* directory and copied the application files to it. We can now navigate into the staging area to create the Ops configuration file.

***Step 1: Prepare* config.json**

As discussed previously, this *config.json* contains various options and attributes of code execution. For our application, we will create *config.json* within the staging area and will pass it as a parameter (*ops --config config.json)* to Ops on execution. Use the commands